VCGen

Boogie Style

Idea

Boogie-style VCGen uses backward safety equations over the CFG.

For each block `i`, introduce:

$$\mathrm{ok}_i$$

`ok_i` means: starting execution at block `i` is safe.

The counterexample query is:

$$\neg \mathrm{ok}_{\mathrm{entry}}$$

No path selection, no block-reachability variables — control flow becomes recursion.

Block Equation

$$\mathrm{ok}_i \Leftrightarrow \left(\mathrm{defs}_i \Rightarrow \left(\mathrm{asserts}_i \wedge \wedge_{j \in \mathrm{succ}(i)} \left(\left(\mathrm{cond}_{i,j} \wedge \mathrm{dsa}_{i,j}\right) \Rightarrow \mathrm{ok}_j\right)\right)\right)$$

- assumptions and definitions are premises
- assertions are consequents
- each feasible successor must be safe
- infeasible blocks are safe vacuously

Terminal Block

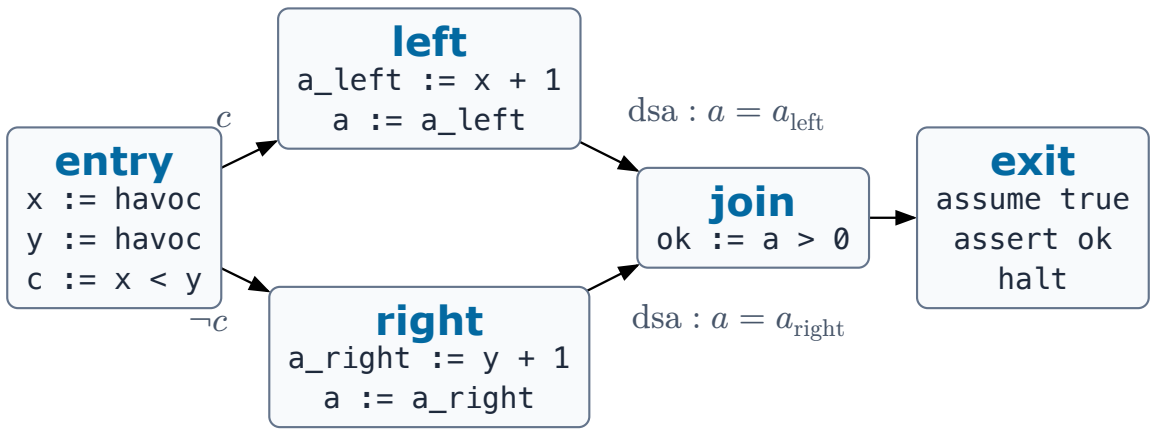
For a terminal block, the successor conjunction is `true`:

$$\mathrm{ok}_i \Leftrightarrow (\mathrm{defs}_i \Rightarrow \mathrm{asserts}_i)$$

A false reachable assertion makes the local `ok_i` false, then unsafety propagates backward to `entry`.

The Diamond, In DSA Form

Boogie-style consumes DSA: the merge moves onto the incoming edges.



The assignments to `a` are **edge-sensitive DSA definitions** — they enter the formula on the edge premises, not as block facts.

Encoded Equations

$$\mathbf{OK}_{\text{entry}} : \text{ok}_{\text{entry}} \Leftrightarrow \big(c = (x < y) \Rightarrow \big((c \Rightarrow \text{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \text{ok}_{\text{right}})\big)\big)$$

$$\mathbf{OK}_{\text{left}} : \text{ok}_{\text{left}} \Leftrightarrow \big(a_{\text{left}} = x + 1 \Rightarrow \big((a = a_{\text{left}}) \Rightarrow \text{ok}_{\text{join}}\big)\big)$$

$$\mathbf{OK}_{\text{right}} : \text{ok}_{\text{right}} \Leftrightarrow \big(a_{\text{right}} = y + 1 \Rightarrow \big((a = a_{\text{right}}) \Rightarrow \text{ok}_{\text{join}}\big)\big)$$

Encoded Exit

$$\mathbf{OK}_{\text{join}} : \text{ok}_{\text{join}} \Leftrightarrow (\text{ok} = (a > 0) \Rightarrow \text{ok}_{\text{exit}})$$

$$\mathbf{OK}_{\text{exit}} : \text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$$

$$\mathbf{DISCHARGE} : \neg\text{ok}_{\text{entry}}$$

The model chooses havoc values and a branch whose successor is unsafe.

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2, \quad y = 0, \quad a = a_{\text{left}} = -1, \quad \text{ok} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2, \quad y = 0, \quad a = a_{\text{left}} = -1, \quad \text{ok} = \text{false}$
- OK_{exit}: $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2, \quad y = 0, \quad a = a_{\text{left}} = -1, \quad \text{ok} = \text{false}$
- OK_{exit}: $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join}: premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\longrightarrow \text{ok}_{\text{join}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2, \quad y = 0, \quad a = a_{\text{left}} = -1, \quad \text{ok} = \text{false}$
- OK_{exit}: $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join}: premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\longrightarrow \text{ok}_{\text{join}} = \text{false}$
- OK_{left}: premises $a_{\text{left}} = x + 1 = -1$ and edge definition $a = a_{\text{left}}$ hold $\longrightarrow \text{ok}_{\text{left}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2, \quad y = 0, \quad a = a_{\text{left}} = -1, \quad \text{ok} = \text{false}$
- OK_{exit}: $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join}: premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\longrightarrow \text{ok}_{\text{join}} = \text{false}$
- OK_{left}: premises $a_{\text{left}} = x + 1 = -1$ and edge definition $a = a_{\text{left}}$ hold $\longrightarrow \text{ok}_{\text{left}} = \text{false}$
- OK_{entry}: $c = (x < y) = \text{true}$, and the true branch requires $c \Rightarrow \text{ok}_{\text{left}} \longrightarrow \text{ok}_{\text{entry}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2, \quad y = 0, \quad a = a_{\text{left}} = -1, \quad \text{ok} = \text{false}$
- OK_{exit}: $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join}: premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\longrightarrow \text{ok}_{\text{join}} = \text{false}$
- OK_{left}: premises $a_{\text{left}} = x + 1 = -1$ and edge definition $a = a_{\text{left}}$ hold $\longrightarrow \text{ok}_{\text{left}} = \text{false}$
- OK_{entry}: $c = (x < y) = \text{true}$, and the true branch requires $c \Rightarrow \text{ok}_{\text{left}} \longrightarrow \text{ok}_{\text{entry}} = \text{false}$
- **DISCHARGE** $\neg\text{ok}_{\text{entry}}$ is satisfied — the model **is** the failing execution `entry -> left -> join -> exit`.

Static ITE Merge Variant

The merge can be compiled into an ordinary definition:

```
join:
  a := ite(c, a_left, a_right)
  ok := a > 0
  goto exit
```

$$\mathrm{ok}_{\mathrm{join}} \Leftrightarrow \Big(\big(a = \mathrm{ite}\big(c, a_{\mathrm{left}}, a_{\mathrm{right}}\big) \wedge \mathrm{ok} = (a > 0)\big) \Rightarrow \mathrm{ok}_{\mathrm{exit}}\Big)$$

Now `left` and `right` have no edge definitions — the predecessor-sensitive merge became a local fact of `join`.

Tradeoffs

Advantages:

- no separate path-selection formula
- many assertions are natural
- assumptions are handled by implication

Costs:

- formula is nested
- cross-block simplification is harder
- phi/ITE extensions need exclusivity side conditions

Phi Extension Needs CFG Facts

If a phi is compiled as:

$$a = \mathrm{ite}\big(\mathsf{bb}_{\mathrm{left}}, a_{\mathrm{left}}, a_{\mathrm{right}}\big)$$

then the block variables used by the ITE must be consistent:

$$\mathsf{bb}_{\mathrm{join}} \Rightarrow \mathsf{bb}_{\mathrm{left}} \vee \mathsf{bb}_{\mathrm{right}}$$

$$\neg\big(\mathsf{bb}_{\mathrm{left}} \wedge \mathsf{bb}_{\mathrm{right}}\big)$$

The backward equations prove safety; the CFG facts keep the ITE merge faithful — the same exclusivity pitfall as SeaHorn's phi-as-ITE.

Same Diamond, Two Formulas

SeaHorn — flat conjunction:

$$\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{exit}} \wedge \ldots$$
$$\mathtt{bb}_{\text{entry}} \Rightarrow c = (x < y)$$
$$\mathtt{bb}_{\text{left}} \Rightarrow a_{\text{left}} = x + 1$$
$$\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{left}}\right) \Rightarrow c$$
$$\left(\mathtt{bb}_{\text{left}} \wedge \mathtt{bb}_{\text{join}}\right) \Rightarrow a = a_{\text{left}}$$
$$\mathtt{bb}_{\text{exit}} \Rightarrow \neg\ \mathrm{ok}$$

facts at top level; path selected by block variables

Boogie — nested recursion:

$$\neg \mathrm{ok}_{\text{entry}}$$
$$\mathrm{ok}_{\text{entry}} \Leftrightarrow \big(c = (x < y) \Rightarrow$$
$$\big((c \Rightarrow \mathrm{ok}_{\text{left}}) \wedge \big(\neg c \Rightarrow \mathrm{ok}_{\text{right}}\big)\big)\big)$$
$$\mathrm{ok}_{\text{left}} \Leftrightarrow \big(a_{\text{left}} = x + 1 \Rightarrow$$
$$\big((a = a_{\text{left}}) \Rightarrow \mathrm{ok}_{\text{join}}\big)\big)$$
$$\ldots$$

facts nested under equations; path selected by implication chains

Same models, same verdict — different levers for the solver: SeaHorn exposes equalities for substitution; Boogie gets assertions and assumptions for free.